Universidad del Valle de Guatemala

Facultad de Ingeniería

Catedrático: Kevin Antonio Velásquez Aguilar

Programación Orientada a Objetos

Ejercicio 6. Polimorfismo vía interfaces

Nombre: María Isabel Cajas Cuéllar

Carné: 26183

Sección: 20

Fecha de entrega: 08/11/2026

Descripción del problema

Una plataforma de comercio electrónico necesita administrar integraciones con distintos servicios externos: pasarelas de pago, proveedores logísticos y certificadores de Factura Electrónica en Línea (FEL). Cada servicio ofrece capacidades diferentes. Algunos procesan cobros y reintegros; otros permiten consultar estados; y otros generan documentos electrónicos.

El programa propone un catálogo central para registrar y administrar estas integraciones. El administrador utiliza un menú de consola para listarlas, buscar una por su identificador, ordenarlas por latencia y probar operaciones. Antes de ejecutar una operación, el sistema comprueba si la integración ofrece esa capacidad.

Objetivo general

Desarrollar un prototipo para administrar integraciones de distintos tipos mediante herencia, interfaces y polimorfismo, utilizando una única lista polimórfica y separando responsabilidades con el patrón MVC.

Objetivos específicos

- Representar los datos comunes de las integraciones en una clase base.
- Definir las capacidades de los servicios mediante interfaces.
- Guardar diferentes tipos de integración en una sola lista.
- Cargar integraciones de ejemplo al iniciar el programa.
- Permitir listar, buscar y ordenar las integraciones.
- Ignorar las solicitudes de operaciones incompatibles sin detener la ejecución.
- Aplicar Comparable para ordenar por latencia promedio.
- Separar la vista, el controlador y el modelo.

Alcance del sistema

El programa carga diez integraciones de ejemplo: cinco pasarelas de pago, tres proveedores logísticos y dos certificadores FEL. Permite consultar sus datos, configuración, estado de conexión y latencia, además de simular operaciones.

Las operaciones no se conectan realmente con bancos, proveedores de envíos ni certificadores FEL. El programa tampoco procesa pagos reales, genera documentos oficiales ni utiliza una base de datos. Los datos se almacenan en memoria y se pierden al cerrar la aplicación. La interfaz implementada es de consola.

Actor del sistema

Actor	Descripción
Administrador	Utiliza el menú para consultar el catálogo, buscar integraciones, ordenar por latencia y probar las operaciones disponibles.

Requisitos funcionales

Código	Requisito
RF1	Cargar integraciones de ejemplo al iniciar el programa.
RF2	Listar las integraciones y mostrar su información y estado de conexión.
RF3	Buscar una integración por su ID y mostrar su configuración.
RF4	Ordenar el catálogo por latencia promedio de menor a mayor.
RF5	Permitir cobros y reintegros en integraciones que ofrezcan operaciones transaccionales.
RF6	Permitir la generación de documentos en integraciones que tengan capacidad documental.
RF7	Consultar el estado y registrar actividades para integraciones auditables.
RF8	Ignorar una solicitud de reintegro cuando la integración no admite transacciones, sin detener el programa.
RF9	Presentar las opciones mediante un menú de consola y permitir salir del programa.

Interfaces

Interfaz	Métodos	Clases que la implementan	Función
Transaccional	cobrar(monto); reintegrar(monto)	PasarelaPago	Define operaciones relacionadas con

			cobros y reintegros.
Documentable	generalDocumento(referencia)	CertificadorFEL	Define la generación simulada de documentos FEL.
Auditable	consultarEstado(); registrarActividad(actividad)	PasarelaPago; ProveedorLogistico; CertificadorFEL	Define la consulta del estado y el registro de actividad.

Analisis de clases

Clase	Atributos	Métodos principales y función
IntegracionAPI	id; proveedor; conectada; latenciaPromedio; configuracion	Clase abstracta que reúne los datos comunes. Declara getTipo(), compara latencias con compareTo() y presenta o compara objetos con toString(), equals() y hashCode().
PasarelaPago	Hereda los atributos de IntegracionAPI	Implementa las interfaces Transaccional y Auditable. Simula cobros y reintegros; consulta el estado y registra actividad.
ProveedorLogistico	Hereda los atributos de IntegracionAPI	Implementa Auditable. Consulta el estado de conexión y registra actividad.
CertificadorFEL	Hereda los atributos de IntegracionAPI	Implementa Documentable y Auditable. Simula la

		generación de documentos; consulta el estado y registra actividad.
CatalogoIntegraciones	integraciones: List<IntegracionAPI>	Administra la lista polimórfica. Ofrece agregar(), getIntegraciones(), buscarPorId() y ordenarPorLatencia().
CargaInicial	No declara atributos de instancia	cargar(catalogo) agrega los ejemplos al catálogo. crearConfiguracion(ambiente, region) crea el mapa de configuración de cada integración.
ServicioOperaciones	No declara atributos	Sobrecarga procesar() para recibir Transaccional, Documentable o Auditable. También ofrece reintegrar().
ControladorIntegraciones	catalogo; servicio	Recibe las solicitudes de la vista, consulta el catálogo, verifica las capacidades de cada integración y delega las operaciones.
MenuConsola	controlador; scanner	Muestra el menú, recibe los datos y presenta resultados. Incluye métodos para listar, buscar, probar operaciones y leer números.
App	No declara atributos de instancia	El método main() crea el catálogo, carga los datos y conecta el controlador con el menú.

Lista polimórfica

Atributo	Tipo	Objetos que puede contener
integraciones	List<IntegracionAPI>	Objetos de tipo PasarelaPago, ProveedorLogistico y CertificadorFEL.

Herencia, interfaces y polimorfismo

Las clases PasarelaPago, ProveedorLogistico y CertificadorFEL heredan de IntegracionAPI, por lo que reutilizan sus atributos y comportamientos comunes. Cada clase define su propio tipo mediante getTipo().

Las interfaces determinan qué operaciones puede ofrecer cada objeto. En el controlador se utiliza instanceof para verificar si una integración implementa la interfaz correspondiente antes de ejecutar la operación. Por ejemplo, un reintegro se procesa solo si el objeto es Transaccional. Si no lo es, se muestra un mensaje y el programa continúa.

Sobrecarga de métodos

Método	Parámetros	Operación
procesar(...)	Transaccional, double	Realiza un cobro.
procesar(...)	Documentable, String	Solicita la generación de un documento.
procesar(...)	Auditable, String	Consulta el estado y registra actividad.

Ordenamiento y buenas prácticas

IntegracionAPI implementa Comparable<IntegracionAPI>. Su método compareTo() compara las latencias promedio. El catálogo utiliza Collections.sort() para ordenar la lista de menor a mayor latencia.

Los atributos de la clase base son privados, lo que aplica encapsulación. Los getters permiten consultar la información. Además, toString() presenta los datos del objeto y equals() junto con hashCode() permiten comparar integraciones según la clase concreta y el ID.

Aplicación del patrón MVC

Componente	Clases	Responsabilidad

Modelo	IntegracionAPI, las clases derivadas, las interfaces, CatalogoIntegraciones, CargaInicial, ServicioOperaciones	Conserva los datos y las reglas de negocio.
Vista	MenuConsola	Presenta las opciones, recibe entradas y muestra resultados.
Controlador	ControladorIntegraciones	Conecta las solicitudes de la vista con el modelo y valida qué operaciones se pueden realizar.
Inicio de la aplicación	App	Crea los componentes y pone en marcha el sistema.

Flujo de ejecutación

1. App crea un catálogo.
2. CargaInicial instancia las integraciones y solicita agregarlas al catálogo.
3. App crea el controlador y la vista.
4. MenuConsola presenta las opciones al administrador.
5. El administrador selecciona una acción e ingresa los datos necesarios.
6. El controlador consulta el catálogo y verifica las capacidades de la integración.
7. ServicioOperaciones ejecuta la operación correspondiente.
8. El resultado se presenta en la consola.

Validaciones y restricciones

- Los métodos de cobro y reintegro rechazan montos menores o iguales a cero.
- El catálogo busca si un ID ya existe antes de agregar una integración.
- La búsqueda por ID distingue entre mayúsculas y minúsculas porque utiliza equals().
- La configuración se devuelve mediante un mapa no modificable desde getConfiguracion().
- Las operaciones dependen de las interfaces implementadas por cada integración.

- El programa muestra el estado de conexión, pero el código no impide ejecutar operaciones cuando una integración aparece desconectada.
- Si el administrador introduce texto donde se espera un número, el menú vuelve a solicitar la entrada.

Evidencia de funcionamiento

https://drive.google.com/drive/folders/1_3lp8QG8mqSArQpDr9kOrx_zcshzBtbx?usp=drive_link

Diagrama UML

https://drive.google.com/file/d/1Cv1QvBvAOo2ot_YZrnY0XfLNsPHQdSHi/view?usp=sharing